

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 1

Introduction

Welcome to CS 213

Course Logistics

- Every week
 - Three lectures
 - One lab
 - Exercises for practice
- Learning Management System: Bodhitree
 - Lab submissions
 - Slides will be available soon after/before lecture
- Evaluation plan (subject to change)
 - Evaluation for CS 213:
 - Participation (10%)
 - Two quizzes (15% x 2)
 - Two exams (25% + 35%)
 - Evaluation for CS 293:
 - Weekly lab evaluation (20%)
 - Two lab quizzes (15% x 2)
 - Two exams (20% + 30%)
- More logistics in the coming days!

Academic Integrity

Do not copy, consult or share

in quizzes, exams and evaluated lab submissions!

Do not copy or use LLMs in evaluated lab submissions.

You can consult references provided, and ask TAs for guidance/hints.

But blindly copying defeats the purpose of the exercises.

All code submitted should be written by yourself

Any suspected cheating will be reported to D-ADAC

What is in store in CS 213/293?

Data Structures and (related) Algorithms

- In 2017, Instagram switched from "BTrees" to "LSM Trees", for storing activity feed
- What are these "trees"? And why did they matter?
 - BTrees data structure, invented in 1970, is a gold standard in databases. (We will see this later in the course.)
 - LSM Trees (invented in 1996) is specialised for exploiting higher sequential write-speeds available in hardware. Turned out to be an important consideration at Instagram's subsequent scale.

Data Structures and (related) Algorithms

- In almost every program dealing with large amounts of data, a careful choice of data structures is crucial for efficiency
 - Many "standard" data structures are available as part of libraries
- In this course:
 - What, why and how of many standard data structures
 - What: lists, queues, trees, hash tables, ...
 - Why: Their efficiency features/limitations
 - How: Their internal workings
 - Some algorithms for data manipulation (sorting, compression, ...)

A Quick CS 101 Recap

A C++ Program

```
#include <iostream>

int main() {
    std::cout << "Hello Data Structures!" << std::endl;
    return 0;
}
```


Another C++ Program



Demo

```
#include <iostream>

struct node {
    std::string word;
    node* next = nullptr;
    void print() {
        std::cout << word << " ";
        if(next) next->print();
        else std::cout << std::endl;
    }
};
```

```
int main() {
    node* first = new node{"Hello"};
    first->next = new node{"World!"};
    first->print();
    return 0;
}
```

Let's Get Started

With a Little Experiment

Store and Retrieve

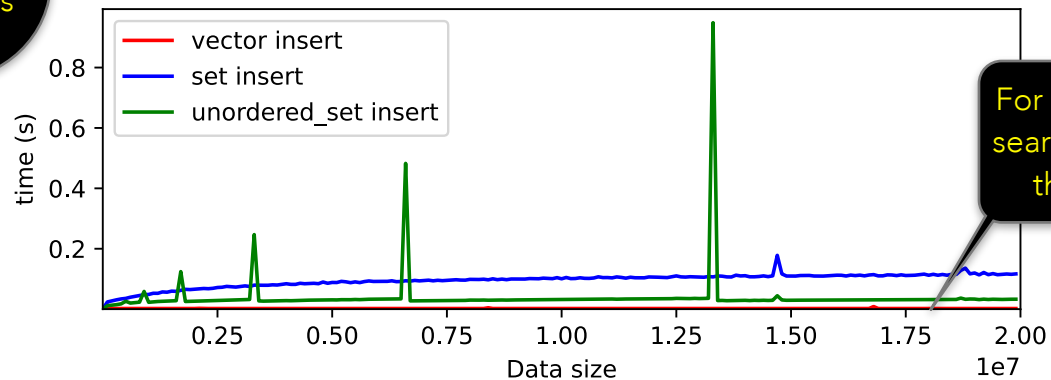
- Task: Store a bunch of integers in a data structure, and later given a query, check if it was already stored
- C++ provides convenient data structures for this, with a "find" method
 - `std::vector`
 - `std::set`
 - `std::unordered_set`
- Let's find out how they each perform

Store and Retrieve

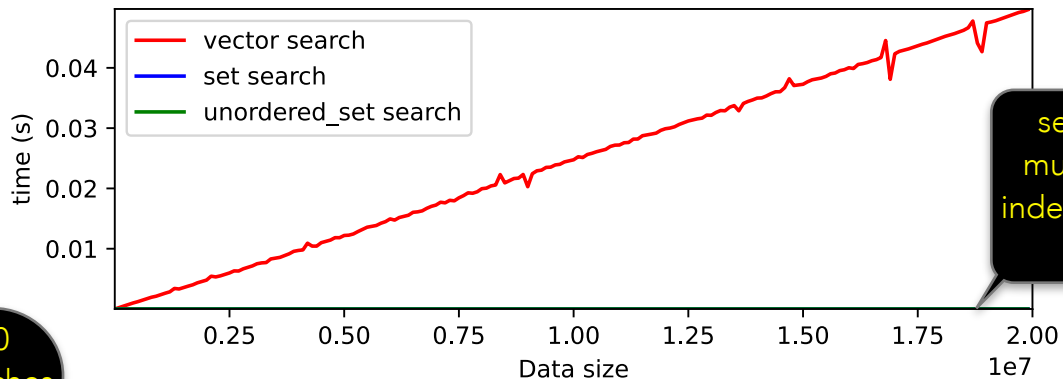
```
std::vector<int> v;  
std::set<int> s;  
std::unordered_set<int> u;  
  
...  
// in a loop  
    v.push_back(item);  
    s.insert(item);  
    u.insert(item);  
  
...  
std::find(v.begin(), v.end(), query); //iterate through the range  
s.find(query);  
u.find(query);
```

Store and Retrieve

100000
insertions



For vector, insert is very fast, but search gets slower and slower as the number of items grows



set and unordered_set have much faster search, seemingly independent of how big the data set is

10
searches

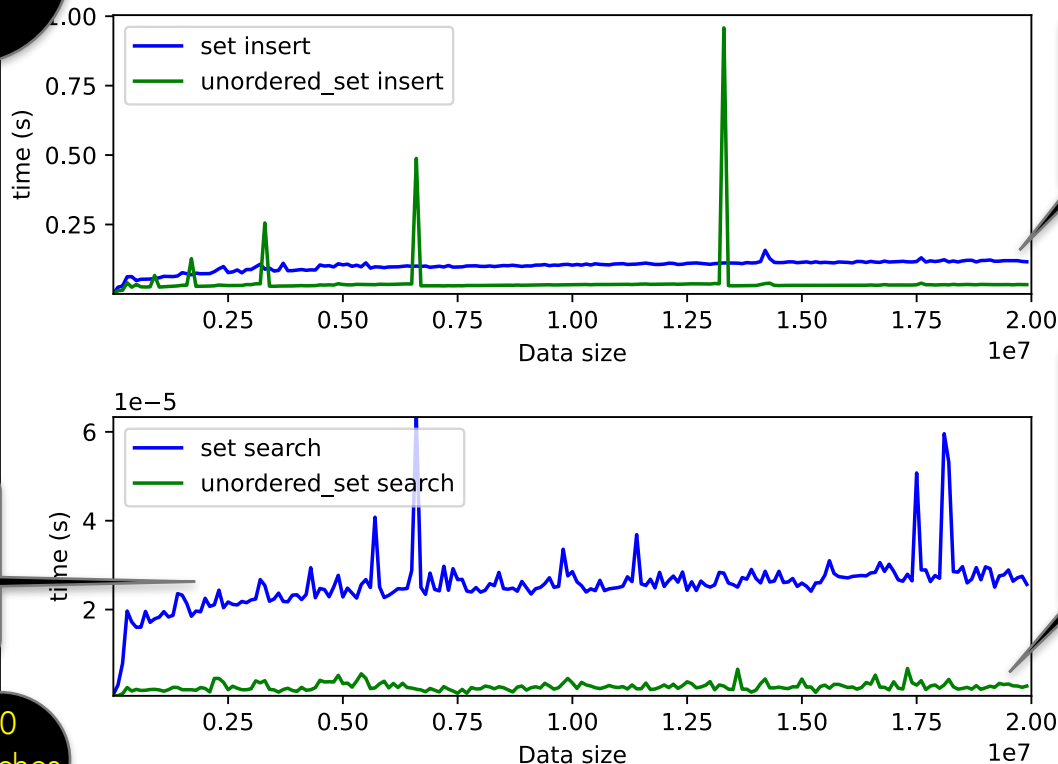
Store and Retrieve

100000
insertions

A closer look at
set vs.
unordered_set

set search is slowly
climbing with the data size

10
searches



set insert is also slowly
climbing with the data size

unordered_set insert and
search both take time
almost independent of the
data-size (except for insert
times spiking at various
data sizes)

Data Structures and (related) Algorithms

- What is under the hood of `std::set` and `std::unordered_set`, and why do they behave so differently from each other?
- Looking ahead:
 - `std::set` is a “balanced binary search tree”
 - `std::unordered_set` is a “hash table”